

ChatGPT - A08_CVE-2023-49297

Here is a snippet of code written in python. I want to check for any potential security vulnerabilities.

```
from yaml import load
from yaml import YAMLError

try:
    from yaml import CLoader as Loader
except ImportError:
    from yaml import Loader

SETTINGS_FILE = "settings.yaml"
SETTINGS_STRUCT = {
    "client_config_backend": {
        "type": str,
        "required": True,
        "default": "file",
        "dependency": [
            {"value": "file", "attribute": ["client_config_file"]},
            {"value": "settings", "attribute": ["client_config"]},
            {"value": "service", "attribute": ["service_config"]},
        ],
    },
    "save_credentials": {
        "type": bool,
        "required": True,
        "default": False,
        "dependency": [
            {"value": True, "attribute":
["save_credentials_backend"]}
        ],
    },
    "get_refresh_token": {"type": bool, "required": False,
"default": False},
    "client_config_file": {
        "type": str,
        "required": False,
        "default": "client_secrets.json",
    },
}
```

```

"save_credentials_backend": {
  "type": str,
  "required": False,
  "dependency": [
    {"value": "file", "attribute": ["save_credentials_file"]},
    {"value": "dictionary", "attribute":
["save_credentials_dict"]},
    {"value": "dictionary", "attribute":
["save_credentials_key"]},
  ],
},
"client_config": {
  "type": dict,
  "required": False,
  "struct": {
    "client_id": {"type": str, "required": True},
    "client_secret": {"type": str, "required": True},
    "auth_uri": {
      "type": str,
      "required": True,
      "default":
"https://accounts.google.com/o/oauth2/auth",
    },
    "token_uri": {
      "type": str,
      "required": True,
      "default":
"https://accounts.google.com/o/oauth2/token",
    },
    "redirect_uri": {
      "type": str,
      "required": True,
      "default": "urn:ietf:wg:oauth:2.0:oob",
    },
    "revoke_uri": {"type": str, "required": True, "default":
None},
  },
},
"service_config": {
  "type": dict,
  "required": False,
  "struct": {

```

```

        "client_user_email": {
            "type": str,
            "required": True,
            "default": None,
        },
        "client_service_email": {"type": str, "required": False},
        "client_pkcs12_file_path": {"type": str, "required":
False},
        "client_json_file_path": {"type": str, "required": False},
        "client_json_dict": {
            "type": dict,
            "required": False,
            "struct": {},
        },
        "client_json": {"type": str, "required": False},
    },
    "oauth_scope": {
        "type": list,
        "required": True,
        "struct": str,
        "default": ["https://www.googleapis.com/auth/drive"],
    },
    "save_credentials_file": {"type": str, "required": False},
    "save_credentials_dict": {"type": dict, "required": False,
"struct": {}},
    "save_credentials_key": {"type": str, "required": False},
}

```

```

class SettingsError(IOError):
    """Error while loading/saving settings"""

```

```

class InvalidConfigError(IOError):
    """Error trying to read client configuration."""

```

```

def LoadSettingsFile(filename=SETTINGS_FILE):
    """Loads settings file in yaml format given file name.

    :param filename: path for settings file. 'settings.yaml' by

```

```

default.
:type filename: str.
:raises: SettingsError
"""

try:
    with open(filename) as stream:
        data = load(stream, Loader=Loader)
except (YAMLError, OSError) as e:
    raise SettingsError(e)
return data


def ValidateSettings(data):
    """Validates if current settings is valid.

    :param data: dictionary containing all settings.
    :type data: dict.
    :raises: InvalidConfigError
    """
    _ValidateSettingsStruct(data, SETTINGS_STRUCT)


def _ValidateSettingsStruct(data, struct):
    """Validates if provided data fits provided structure.

    :param data: dictionary containing settings.
    :type data: dict.
    :param struct: dictionary containing structure
    information of settings.
    :type struct: dict.
    :raises: InvalidConfigError
    """
    # Validate required elements of the setting.
    for key in struct:
        if struct[key]["required"]:
            _ValidateSettingsElement(data, struct, key)


def _ValidateSettingsElement(data, struct, key):
    """Validates if provided element of settings data fits
    provided structure.

```

```

:param data: dictionary containing settings.
:type data: dict.
:param struct: dictionary containing structure
information of settings.
:type struct: dict.
:param key: key of the settings element to validate.
:type key: str.
:raises: InvalidConfigError
"""

# Check if data exists. If not, check if default value exists.
value = data.get(key)
data_type = struct[key]["type"]
if value is None:
    try:
        default = struct[key]["default"]
    except KeyError:
        raise InvalidConfigError("Missing required setting
%s" % key)
    else:
        data[key] = default
# If data exists, Check type of the data
elif not isinstance(value, data_type):
    raise InvalidConfigError(f"Setting {key} should be type
{data_type}")
# If type of this data is dict, check if structure of the data
is valid.
if data_type is dict:
    _ValidateSettingsStruct(data[key], struct[key]["struct"])
# If type of this data is list, check if all values in the list is
valid.
elif data_type is list:
    for element in data[key]:
        if not isinstance(element, struct[key]["struct"]):
            raise InvalidConfigError(
                "Setting %s should be list of %s"
                % (key, struct[key]["struct"])
            )
# Check dependency of this attribute.
dependencies = struct[key].get("dependency")
if dependencies:
    for dependency in dependencies:
        if value == dependency["value"]:

```

```
for reqkey in dependency["attribute"]:  
    _ValidateSettingsElement(data, struct, reqkey)
```

Here are **potential security vulnerabilities** and **areas for improvement** in the provided Python code:

1. Unsafe `yaml.load` Use

- **Issue:** `yaml.load` is unsafe when used with untrusted input because it can execute arbitrary code, leading to **Remote Code Execution (RCE)**.
- **Solution:** Use `yaml.safe_load` instead, which avoids this risk.

Fix:

```
python  
  
from yaml import safe_load as load # Use safe_load instead of load
```

2. Missing File Permission Handling

- **Issue:** The `open()` function doesn't specify permissions. If the `settings.yaml` file contains sensitive information (e.g., API keys or secrets), this could expose it to unauthorized access.
- **Solution:** Set the appropriate permissions for the file on the operating system or use Python's `os` module to ensure safe access.

Recommendation:

- Use `os.open()` with explicit permissions (`os.O_RDONLY`).
- Confirm that the YAML file is not world-readable:

```
bash  
  
chmod 600 settings.yaml
```

3. Dependency Handling Without Validation

- **Issue:** If the dependencies are not properly validated, incorrect configurations might bypass validation. The nested dependency logic could become brittle or insecure.
- **Solution:** Add logging or strict validation to catch missing dependencies earlier in the process.

Suggestion:

- Log warnings or errors when a required dependency is missing:

```
python

import logging
logging.basicConfig(level=logging.WARNING)

def _ValidateSettingsElement(data, struct, key):
    # Dependency validation with logging
    ...
    if dependencies:
        for dependency in dependencies:
            if value == dependency["value"]:
                for reqkey in dependency["attribute"]:
                    if reqkey not in data:
                        logging.warning(f"Missing dependency: {reqkey}")
                        raise InvalidConfigError(f"Missing dependency:
{reqkey}")
```

4. Hardcoded Default Credentials and Sensitive Data

- **Issue:** Hardcoding sensitive URLs (like OAuth token URIs) or default paths can lead to information leakage.
- **Solution:** Move these URLs and sensitive information to environment variables or configuration files that are not hardcoded in the source code.

Recommendation:

```
python
```

```
import os
```

```
AUTH_URI = os.getenv("AUTH_URI", "https://accounts.google.com/o/oauth2/auth")
```

5. Error Handling May Leak Sensitive Information

- **Issue:** The raised exceptions might expose sensitive information about the system (e.g., the reason for failure).
- **Solution:** Avoid raising raw exceptions. Instead, sanitize the error messages to prevent leakage of information.

Fix:

```
python
```

```
def LoadSettingsFile(filename=SETTINGS_FILE):
```

```
    try:
```

```
        with open(filename) as stream:
```

```
            data = load(stream, Loader=Loader)
```

```
    except (YAMLError, OSError) as e:
```

```
        raise SettingsError("Failed to load settings file. Please check the file.")
```

```
    return data
```

6. Lack of Input Sanitization

- **Issue:** There is no input sanitization when loading or validating the YAML configuration. Malicious inputs could pass through and cause issues later in the application.
- **Solution:** Implement checks to validate expected inputs thoroughly.

7. Potential Configuration Injection

- **Issue:** The YAML file configuration could be tampered with to inject unwanted behaviors (e.g., swapping API URIs).
 - **Solution:** Validate or sign the configuration file to prevent unauthorized changes.
-

Final Recommendation

Here is a **refactored** `LoadSettingsFile` function addressing the critical issue (unsafe YAML loading):

```
python

from yaml import safe_load, YAMLError

def LoadSettingsFile(filename=SETTINGS_FILE):
    """Loads settings file in YAML format given the file name."""
    try:
        with open(filename, "r") as stream:
            data = safe_load(stream) # Use safe_load for safety
    except (YAMLError, OSError) as e:
        raise SettingsError("Failed to load settings file: {}".format(str(e)))
    return data
```

Addressing these vulnerabilities ensures that the code is more **secure, maintainable**, and **resilient to attacks**.

Provide the exact part of the code along with line numbers that are vulnerable or have security issues.

Below are the **specific parts of the code with line numbers** that contain **vulnerabilities or security issues**, along with the explanation for each issue:

1. Unsafe Use of `yaml.load` (Line 96)

Code:

```
python

96     data = load(stream, Loader=Loader)
```

Issue:

- Using `yaml.load` is insecure, as it can execute arbitrary code from the YAML input. This opens the code to **Remote Code Execution (RCE)** if the input is malicious.

Fix:

```
python

96     data = safe_load(stream)  # Use safe_load instead of load
```

2. Missing File Permission Handling (Line 95)

Code:

```
python

95     with open(filename) as stream:
```

Issue:

- No file permission is explicitly set. If the file contains sensitive data (like secrets or tokens), it may be accessed by unauthorized users.

Recommendation:

Set permissions using `os.open()` or ensure the YAML file is readable only by authorized users (e.g., `chmod 600 settings.yaml` on Unix).

3. Hardcoded Default Credentials and Sensitive Data (Lines 41-66)

Code:

```
python

41     "client_id": {"type": str, "required": True},
42     "client_secret": {"type": str, "required": True},
46     "auth_uri": {
47         "type": str,
48         "required": True,
49         "default": "https://accounts.google.com/o/oauth2/auth",
50     },
54     "token_uri": {
55         "type": str,
56         "required": True,
57         "default": "https://accounts.google.com/o/oauth2/token",
58     },
```

Issue:

- Hardcoding sensitive data (like OAuth URLs, tokens, or paths) increases the risk of **information leakage** if the source code is exposed.

Recommendation:

```
python

import os

AUTH_URI = os.getenv("AUTH_URI", "https://accounts.google.com/o/oauth2/auth")
TOKEN_URI = os.getenv("TOKEN_URI", "https://accounts.google.com/o/oauth2/token")
```

4. Improper Error Handling (Line 98)

Code:

```
python
```

```
98     except (YAMLError, OSError) as e:
99         raise SettingsError(e)
```

Issue:

- Exposing the raw exception `e` may leak sensitive information about the file system or the environment, which can be exploited in an attack.

Fix:

python

```
98     except (YAMLError, OSError) as e:
99         raise SettingsError("Failed to load settings file. Please check the
file.")
```

5. Weak Dependency Validation Logic (Line 143)

Code:

python

```
143     dependencies = struct[key].get("dependency")
144     if dependencies:
145         for dependency in dependencies:
146             if value == dependency["value"]:
147                 for reqkey in dependency["attribute"]:
148                     _ValidateSettingsElement(data, struct, reqkey)
```

Issue:

- If the dependencies are not correctly validated, incorrect configurations could lead to unexpected behavior or application misconfigurations.

Recommendation: Add logging for missing dependencies to detect such issues:

python

```
import logging

145     for dependency in dependencies:
146         if value == dependency["value"]:
147             for reqkey in dependency["attribute"]:
148                 if reqkey not in data:
149                     logging.warning(f"Missing dependency: {reqkey}")
150                     raise InvalidConfigError(f"Missing dependency: {reqkey}")
```

6. No Input Sanitization in YAML Parsing (Line 96)

Code:

```
python

96     data = load(stream, Loader=Loader)
```

Issue:

- Although YAML parsing occurs, no additional input validation or sanitization is done. If malicious data gets loaded, it might trigger unwanted behavior.

Fix: Use `safe_load` and perform validation checks on parsed data to ensure the integrity of inputs:

```
python

96     data = safe_load(stream)
97     if not isinstance(data, dict):
98         raise SettingsError("Settings file content is invalid or corrupted.")
```

7. Potential Configuration Injection Risk (Lines 37-38)

Code:

```
python
```

```
37     "client_config_file": {  
38         "type": str, "required": False, "default": "client_secrets.json",
```

Issue:

- If the `client_config_file` or other configuration elements are tampered with, it could result in unintended behavior (e.g., pointing to malicious secrets).

Recommendation:

Validate the integrity of the configuration file, such as by **signing** it or verifying its checksum before loading:

```
python
```

```
import hashlib
```

```
def validate_file_integrity(filename, expected_hash):  
    with open(filename, 'rb') as f:  
        file_hash = hashlib.sha256(f.read()).hexdigest()  
    if file_hash != expected_hash:  
        raise SettingsError("Configuration file integrity check failed.")
```

Summary of Vulnerable Sections with Line Numbers

Line No.	Code Snippet	Issue	Recommended Fix
96	<code>data = load(stream, Loader=Loader)</code>	Unsafe YAML load	Use <code>safe_load</code>
95	<code>with open(filename) as stream:</code>	No file permission set	Use <code>os.open()</code> or proper <code>chmod</code> settings
41-66	Hardcoded OAuth URLs and sensitive data	Information leakage	Use environment variables
98-99	<code>raise SettingsError(e)</code>	Raw error leaking sensitive info	Sanitize error messages

Line No.	Code Snippet	Issue	Recommended Fix
143-148	Weak dependency validation	Configuration inconsistencies	Add logging and stricter checks
37-38	<pre>"default": "client_secrets.json"</pre>	Configuration injection risk	Validate file integrity

These changes will address the identified security issues, improving the **robustness** and **security** of the code.

Please analyze the code for common security vulnerabilities if present and map them to OWASP Top 10 Web Application Security Risk category/categories.

Below is a detailed analysis of the **security vulnerabilities** identified in the provided Python code, mapped to the appropriate **OWASP Top 10 Web Application Security Risk** categories:

1. Insecure Deserialization (A08:2021)

Vulnerable Code (Line 96):

```
python  
  
data = load(stream, Loader=Loader)
```

Explanation:

- Using `yaml.load()` introduces the risk of **insecure deserialization**, where untrusted YAML data can execute arbitrary code during deserialization. This is a classic example of **Remote Code Execution (RCE)**.
- This vulnerability aligns with **OWASP A08:2021 - Software and Data Integrity Failures**, as it allows attackers to inject malicious data leading to code execution.

Solution:

- Replace `yaml.load()` with `yaml.safe_load()` to avoid code execution.

```
python
```

```
data = safe_load(stream)
```

2. Sensitive Data Exposure (A02:2021)

Vulnerable Code (Lines 41-66):

```
python
```

```
"client_id": {"type": str, "required": True},  
"client_secret": {"type": str, "required": True},  
"auth_uri": {"default": "https://accounts.google.com/o/oauth2/auth"},  
"token_uri": {"default": "https://accounts.google.com/o/oauth2/token"},
```

Explanation:

- Hardcoding **sensitive data** like OAuth credentials and URLs exposes them in the source code, which can be exploited if the source code is leaked or compromised.
- This maps to **OWASP A02:2021 - Cryptographic Failures** because improper storage of secrets can lead to credential theft or misuse.

Solution:

- Move sensitive data (such as `client_id` and `client_secret`) to environment variables:

```
python
```

```
import os  
CLIENT_ID = os.getenv("CLIENT_ID")  
CLIENT_SECRET = os.getenv("CLIENT_SECRET")
```


3. Improper Error Handling (A01:2021)

Vulnerable Code (Lines 98-99):

```
python

except (YAMLError, OSError) as e:
    raise SettingsError(e)
```

Explanation:

- Raising raw exceptions without sanitization can **expose internal system details**, such as file paths or YAML parsing errors, which attackers can use to craft targeted attacks.
- This relates to **OWASP A01:2021 - Broken Access Control**, as the error might leak information about the underlying system that shouldn't be accessible to users.

Solution:

- Use generic error messages to avoid exposing sensitive internal details:

```
python

except (YAMLError, OSError):
    raise SettingsError("Failed to load the settings file.")
```

4. Security Misconfiguration (A05:2021)

Vulnerable Code (Line 95):

```
python

with open(filename) as stream:
```

Explanation:

- The YAML file might contain sensitive information, yet there are **no controls on file permissions**. If this file is misconfigured to be readable by unauthorized users, it could lead to data exposure.

- This vulnerability falls under **OWASP A05:2021 - Security Misconfiguration**, which highlights the importance of securing configuration files.

Solution:

- Ensure proper file permissions are enforced:

```
bash

chmod 600 settings.yaml # Ensure the file is readable only by the application.
```

5. Lack of Input Validation (A03:2021)

Vulnerable Code (Line 96):

```
python

data = load(stream, Loader=Loader)
```

Explanation:

- There is no **validation or sanitization** of the loaded data. If malicious data is passed into the YAML configuration, it could corrupt the application state or enable unintended behaviors.
- This issue maps to **OWASP A03:2021 - Injection**, where unvalidated input can lead to security flaws.

Solution:

- After loading the YAML, perform validation checks:

```
python

data = safe_load(stream)
if not isinstance(data, dict):
    raise SettingsError("Invalid configuration format.")
```

6. Insecure Dependency Handling (A06:2021)

Vulnerable Code (Lines 143-148):

```
python

dependencies = struct[key].get("dependency")
if dependencies:
    for dependency in dependencies:
        if value == dependency["value"]:
            for reqkey in dependency["attribute"]:
                _ValidateSettingsElement(data, struct, reqkey)
```

Explanation:

- Dependencies are validated within nested logic, but there is **no proper logging or enforcement** of missing dependencies. If these dependencies are missing, it could result in **unexpected behaviors** or application crashes.
- This aligns with **OWASP A06:2021 - Vulnerable and Outdated Components**, where failure to properly handle dependencies can lead to system misconfigurations.

Solution:

- Log missing dependencies and raise clear exceptions:

```
python

import logging

if dependencies:
    for dependency in dependencies:
        if value == dependency["value"]:
            for reqkey in dependency["attribute"]:
                if reqkey not in data:
                    logging.error(f"Missing required dependency: {reqkey}")
                    raise InvalidConfigError(f"Dependency missing: {reqkey}")
```

7. Configuration Injection Risk (A04:2021)

Vulnerable Code (Lines 37-38):

```
python

"client_config_file": {
    "type": str, "required": False, "default": "client_secrets.json",
}
```

Explanation:

- If the configuration file (`client_config_file`) is modified or injected with malicious content, it could cause the application to load **malicious configurations**.
- This is an example of **OWASP A04:2021 - Insecure Design**, where weak validation or control over configuration files can lead to exploitation.

Solution:

- Implement file integrity checks to ensure the configuration has not been tampered with:

```
python

import hashlib

def validate_file_integrity(filename, expected_hash):
    with open(filename, 'rb') as f:
        file_hash = hashlib.sha256(f.read()).hexdigest()
    if file_hash != expected_hash:
        raise SettingsError("Configuration file integrity check failed.")
```

Mapping to OWASP Top 10 Web Application Security Risks

Risk Category	Vulnerable Code	Description
A01:2021 - Broken Access Control	Line 99	Exposing internal error messages may reveal sensitive system information.
A02:2021 - Cryptographic Failures	Lines 41-66	Hardcoded secrets and credentials increase the risk of sensitive data exposure.

Risk Category	Vulnerable Code	Description
A03:2021 - Injection	Line 96	Lack of input validation allows injection of malicious YAML data.
A04:2021 - Insecure Design	Lines 37-38	Configuration injection due to weak file validation controls.
A05:2021 - Security Misconfiguration	Line 95	No control over file permissions on sensitive configuration files.
A06:2021 - Vulnerable Components	Lines 143-148	Missing dependency validation could cause misconfigurations.
A08:2021 - Software Integrity Failures	Line 96	Insecure deserialization with <code>yaml.load</code> can lead to Remote Code Execution.

Conclusion

The provided code contains several **security vulnerabilities** mapped to multiple **OWASP Top 10 categories**. Addressing these issues by using **safe YAML loading**, **error message sanitization**, **proper dependency management**, and **file integrity validation** will significantly improve the security of the application.